

Collab - Product Requirements Document (PRD)

1. Product Mission

Collab helps hackathon teams, clubs, student communities, and early-stage startups showcase who they are, what they build, and how others can collaborate with them. The product creates a simple public-facing portfolio where teams can present their identity, projects, and work while using secure authentication and role-based permissions for internal collaboration.

Mission Statement

To make team visibility and collaboration effortless by turning a group's work, story, and momentum into a trusted, public digital presence.

2. Product Summary

The current project architecture is a three-layer system:

- Static frontend for public browsing and team pages
- Express backend for authenticated business logic and team management
- Supabase Auth + Postgres for identity, authorization, and row-level security

This design intentionally keeps the browser away from direct database access. The frontend is a public experience, while protected actions are routed through the API layer, which enforces JWT-based authentication and scoped authorization.

Observed Product Direction

From the codebase and documentation, the product's goal is to support:

- Public team profiles and portfolios
- Team creation and owner/admin management
- Member invitations and access control
- Project/work showcases with public/private visibility options
- Secure authenticated profile management
- Safe, centralized handling of uploads and media content

3. Problem Statement

Many teams lack a polished, low-friction way to present themselves online. They often rely on disconnected tools like LinkedIn profiles, GitHub repos, docs, and informal channels that are fragmented and difficult to maintain.

The result is:

- Limited discoverability of team identity and capabilities
- No consistent workspace for public project storytelling
- Weak internal coordination for team leadership and onboarding
- A fragmented experience that makes collaboration harder than it should be

Collab solves this by providing a unified, public-facing presence with secure private collaboration behind the scenes.

4. Target Users

Primary Users

Team Founders / Owners

- Create team profiles
- Publish project work and visible updates
- Invite teammates and assign roles
- Manage public-facing identity and messaging

Team Members

- View team portfolio and access internal team content
- Update personal profile information
- Collaborate on public/private team activities

- Participate in project and work workflows

Visitors / Recruiters / Community Members

- Discover teams and projects
- Review public portfolios, project work, and team bios
- Learn how to reach out or collaborate

Secondary Users

- Hackathon organizers
- Club officers
- Startup accelerators
- Mentors and community partners

5. Product Goals

Business Goals

- Increase visibility for teams and communities
- Create a trusted platform for project discovery
- Support community-driven networking and collaboration
- Build a reusable portfolio engine for teams across multiple contexts

Product Goals

- Make team profiles easy to create and maintain
- Promote public project/portfolio content without exposing internal systems
- Keep member management secure and roles clear
- Support a strong but simple MVP that is easy to extend

Success Metrics

- Team profiles created and maintained over time

- Number of public work items and portfolio entries
- Member activation and invite completion rates
- Repeat use by team owners and leaders
- Public profile visits and engagement with work items

6. User Experience Principles

- Public-first experience: team pages should feel discoverable and professional.
- Secure-by-default: private collaboration must remain protected with role checks and RLS.
- Simple onboarding: team creation and profile setup should be clear and fast.

4. Lightweight trust model: users should understand who can view/edit content. 5. Expandable architecture: the MVP should support future growth without redesign.

7. Core User Flows

7.1 Team Creation

- User signs in through Supabase Auth
- User creates a team with name, slug, and tagline
- System validates slug uniqueness
- System creates team record and assigns owner membership

7.2 Team Profile Viewing

- Public visitors access a Team page
- Team and work items are shown when visibility rules permit
- Authenticated members can access private team content

7.3 Team Membership Management

- Owner/admin can invite members
- Roles define access and permissions
- Membership checks are enforced centrally by backend authorization middleware

7.4 Work Showcase Management

- Team members create work items with title, description, link, visibility, and status
- Public work items are visible to visitors
- Private work items remain restricted to members

7.5 Profile Updates

- Authenticated users can update personal profile information
- System stores profile metadata and keeps public user presentation clean

8. Functional Requirements

8.1 Authentication and Session Management

- Users sign in using Supabase Auth
- Browser requests include bearer tokens to the API
- The API validates JWTs and enforces identity-based access rules
- Role checks and membership checks are applied to protected routes
- Logout and session state must remain consistent with security expectations

8.2 Team Management

- Users can create teams with unique slug URLs
- Team record includes name, slug, tagline, bio, and privacy status
- Owners/admins can update team details
- Team visibility rules must be enforced at the API layer

8.3 Membership and Permissions

- Team members have role-based membership
- Roles should include at least owner, admin, and member
- Sensitive operations require owner/admin permissions
- Membership validation is required before reads, edits, and invites

8.4 Invites

- Team admins should be able to invite collaborators
- Invite workflows should map to role-based onboarding
- Invites must be governed by secure backend authorization

8.5 Posts / Updates

- Teams can publish updates or posts
- Public or member-only visibility should be supported
- Content should be sanitized and validated before persistence

8.6 Work / Portfolio Items

- Teams can create project or work showcase entries
- Entries include title, description, status, primary link, and visibility
- Users can attach metadata and file references where necessary
- Work items must support retrieval with visibility filtering

8.7 Profiles

- Users can update personal profiles, bio, and display fields
- The system should create a profile automatically when a user first accesses the platform
- Public profile information must be minimal and intentionally shaped for portfolio use

8.8 Uploads

- Upload features are intentionally guarded until storage approval is in place
- Upload handling should reject unsupported or invalid content

- Storage flows should be implemented only after operational and security review

9. Non-Functional Requirements

Security

- Only backend services handle direct database access from authenticated users
- JWT claims must be verified securely
- RLS must restrict data access by the authenticated identity
- Authorization checks must be centralized and role-aware
- Input validation and sanitization should be mandatory for all user-provided data

Privacy

- Public team content should be separated from private team coordination
- Personal profile data should be treated as sensitive by default
- Private content should be hidden unless the viewer is authorized

Reliability

- Health checks should confirm the API can reach Postgres
- Errors should not leak database internals to clients
- Logging and audit actions should be retained for operational visibility

Performance

- The platform should support fast public reads for team pages and portfolios
- List operations should be limited to reasonable result sizes
- Public pages should avoid unnecessary private data exposure

Accessibility and UX

- Public pages should be mobile-friendly and readable

- Team content should be easy to scan and navigate
- Basic accessibility standards should be considered as the interface expands

10. MVP Scope

The MVP should include:

- User authentication via Supabase Auth
- Team creation and team detail viewing
- Public/private team profiles
- Member roles and access control enforcement
- Team member management and invite flow
- Work items with visibility toggles
- Public portfolio browsing
- User profile editing
- Basic API security and health monitoring

Out of Scope for MVP

- Advanced file storage and media galleries
- Large-scale community discovery search
- Marketplace or hiring features
- Payment or subscriptions
- Full CMS capability
- Broad internal analytics dashboard

11. Acceptance Criteria

Authentication

- A registered user can sign in and retrieve basic profile information
- The API rejects unauthorized calls to protected routes@
- Authenticated requests include the expected identity context

Team Creation

- A user can create a team with a unique slug
- A duplicate slug is rejected with a clear error
- The creator is assigned owner role automatically

Team Access

- Public team information is visible to anonymous visitors
- Private member-only content remains hidden from non-members
- Access control is consistent between route and database policy layers

Work Items

- A member can create a public or private work item
- Public items are visible to visitors; private items are not
- Only authorized users can delete work items they are allowed to manage

Profile Editing

- Authenticated users can update profile details without breaking data integrity
- Missing or null fields are handled safely and consistently

12. Risks and Constraints

Risks

- Unauthorized access due to inconsistent security checks
- Overexposure of private data in public lists

- Team slug collisions or naming confusion
- Incomplete role modeling if admin rights are not clearly defined

Constraints

- Browser must not call Postgres directly
- Public and private views must be kept intentionally separate
- Upload features require stricter storage review before full activation
- Data and auth architecture should remain simple enough for an MVP

13. Proposed Roadmap

Phase 1: MVP Launch

- Public team pages
- Team creation and membership system
- Work showcase items
- Secure profile management
- Basic protected API and RLS enforcement

Phase 2: Growth Features

- Invite workflows and stronger role operations
- Richer team content and post updates
- Better public discovery and navigation
- Improved profile branding and media support

Phase 3: Scale and Monetization

- Search and recommendations
- Analytics and engagement dashboards

- Enhanced collaboration workflows
- Community and recruiter features

14. Product Recommendation

Collab has a strong MVP foundation: it addresses a real need, uses a security-first architecture, and is built around a compelling public portfolio use case. The strongest path forward is to keep the product focused on team discovery, portfolio showcase, and role-aware collaboration without expanding too early into broad community or marketplace ambitions.

The best next steps are:

- Validate the core team and work portfolio flows with real users
- Tighten invite and role management based on observed usage
- Prioritize public-facing polish and trust signals

4. Introduce storage and richer media only after security review 5. Expand gradually based on evidence from team engagement and usability

15. Appendix: Repository-Derived Product Facts

Based on the current implementation, the system currently includes:

- Static frontend served via HTTP server on port 8080
- Express API with authentication, security middleware, and health endpoints
- Team, member, invite, post, and work routes
- Supabase integration for authentication and database access
- Postgres RLS patterns designed around authenticated identities
- Audit logging for actions such as creation, updates, and deletions

This confirms that the product is already aligned around a portfolio-centric collaboration model rather than a generic social network or CMS.